

Nested Varying Effects

General Principles

To model hierarchies where groups are themselves nested within larger units (e.g., students within classes, which are within schools), we use *Nested Varying Effects*. This allows the model to share information across different levels of the hierarchy, improving estimates through multilevel pooling.

Note

Note

- We have the same considerations as for [Varying Intercepts](#) and [Varying Slopes](#).
- In a nested model, a parameter at one level (e.g., a group-level intercept) becomes the mean for the distribution of parameters at a lower level (e.g., individual-level intercepts).
- This structure is often represented using multiple indices: j for the group and $k(j)$ for the super-group (e.g., region) that group j belongs to.
- To capture the correlation between multiple varying effects (e.g., intercept and slope) at any level of the hierarchy, we use a **Multivariate Normal (MVN)** distribution.
- Non-centered parameterization is highly recommended for nested models to avoid “funnel” geometries that can hinder MCMC sampling.

Example

Below is an example of a nested model with both **varying intercepts** and **varying slopes**. We model the outcome y for individuals in **groups** nested within **regions**. The relationship between x and y varies at both levels.

Python (Raw MVN)

```
from BI import bi
import jax.numpy as jnp
import jax
import numpy as np

# Setup device-----
m = bi(platform='cpu')

# Simulated Data -----
np.random.seed(42)
N_regions = 5
N_groups_per_region = 4
N_groups = N_regions * N_groups_per_region
N_obs_per_group = 20
N = N_groups * N_obs_per_group

# Region level (Top level) parameters
mu_a_region, mu_b_region = 5.0, -1.0
sigma_a_region, sigma_b_region = 1.0, 0.5
rho_region = -0.5

# Group level (Nested level) parameters
sigma_a_group, sigma_b_group = 0.5, 0.2
rho_group = 0.3
sigma_obs = 0.5

# Generate region effects
mu_region = np.array([mu_a_region, mu_b_region])
sigmas_region = np.array([sigma_a_region, sigma_b_region])
Rho_region = np.array([[1, rho_region], [rho_region, 1]])
Sigma_region = np.outer(sigmas_region, sigmas_region) * Rho_region
region_effects = np.random.multivariate_normal(mu_region, Sigma_region, size=N_regions)

# Generate group effects nested in regions
group_to_region = np.repeat(np.arange(N_regions), N_groups_per_region)
sigmas_group = np.array([sigma_a_group, sigma_b_group])
Rho_group = np.array([[1, rho_group], [rho_group, 1]])
Sigma_group = np.outer(sigmas_group, sigmas_group) * Rho_group

group_effects = np.zeros((N_groups, 2))
```

```

for g in range(N_groups):
    reg = group_to_region[g]
    group_effects[g] = np.random.multivariate_normal(region_effects[reg], Sigma_group)

# Generate observations
group_id = np.repeat(np.arange(N_groups), N_obs_per_group)
x = np.random.normal(0, 1, size=N) # 1D
a_g = group_effects[:, 0]
b_g = group_effects[:, 1]
mu = a_g[group_id] + b_g[group_id] * x
y = np.random.normal(mu, sigma_obs) # 1D

m.data_on_model = {
    "group_id": group_id.astype(np.int32),
    "x": x.astype(np.float32),
    "y": y.astype(np.float32),
    "group_to_region": group_to_region.astype(np.int32),
    "N_regions": N_regions,
}

# Define model -----
def model_nested(group_id, x, y, group_to_region, N_regions):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region level
    mu_a_reg = m.dist.normal(5, 2, name='mu_a_reg')
    mu_b_reg = m.dist.normal(-1, 1, name='mu_b_reg')
    sigma_reg = m.dist.exponential(1, shape=(2,), name='sigma_reg')
    Rho_reg = m.dist.lkj(2, 2, name='Rho_reg')
    cov_reg = jnp.outer(sigma_reg, sigma_reg) * Rho_reg

    region_effects = m.dist.multivariate_normal(
        jnp.stack([mu_a_reg, mu_b_reg]),
        cov_reg,
        shape=(N_regions,),
        name='region_effects'
    )

    # 2. Group level
    sigma_grp = m.dist.exponential(1, shape=(2,), name='sigma_grp')
    Rho_grp = m.dist.lkj(2, 2, name='Rho_grp')
    cov_grp = jnp.outer(sigma_grp, sigma_grp) * Rho_grp

```

```

# Nested mean
mu_grp = region_effects[group_to_region]

group_effects = m.dist.multivariate_normal(
    mu_grp,
    cov_grp,
    name='group_effects'
)

a_g_est = group_effects[:, 0]
b_g_est = group_effects[:, 1]

# Likelihood
mu_est = a_g_est[group_id] + b_g_est[group_id] * x
m.dist.normal(mu_est, sigma, obs=y)

# Run sampler -----
m.fit(model_nested, num_samples=1000, num_warmup=500, num_chains=1, progress_bar=False)

# Diagnostic -----
# Generate comparative table (suppressing full summary)
import io
from contextlib import redirect_stdout
with redirect_stdout(io.StringIO()):
    m.summary() # Populates m.tab_summary

print("\n### Parameter Comparison")
print(f"{'Parameter':<15} | {'True':<10} | {'Posterior Mean':<15}")
print("-" * 45)
params_to_check = {
    "mu_a_reg": mu_a_region,
    "mu_b_reg": mu_b_region,
    "sigma_reg[0]": sigma_a_region,
    "sigma_reg[1]": sigma_b_region,
    "sigma": sigma_obs
}

for p, true_val in params_to_check.items():
    if p in m.tab_summary.index:
        post_mean = m.tab_summary.loc[p, "mean"]
        print(f"{p:<15} | {true_val:<10.2f} | {post_mean:<15.2f}")

```

```
jax.local_device_count 16
```

```
arviz - WARNING - Shape validation failed: input_shape: (1, 1000), minimum_shape: (chains=2,  
C:\Users\Sosa\AppData\Local\Programs\Python\Python313\Lib\site-packages\arviz\stats\diagnost
```

```
invalid value encountered in scalar divide
```

```
C:\Users\Sosa\AppData\Local\Programs\Python\Python313\Lib\site-packages\arviz\stats\diagnost
```

```
invalid value encountered in scalar divide
```

```
### Parameter Comparison
```

Parameter	True	Posterior Mean
mu_a_reg	5.00	4.84
mu_b_reg	-1.00	-0.83
sigma_reg[0]	1.00	1.00
sigma_reg[1]	0.50	0.47
sigma	0.50	0.49

Python (Hierarchical Normal)

```
from BI import bi, jnp
import jax
import numpy as np
# Setup device-----
m = bi(platform='cpu')

# Simulated Data -----
np.random.seed(42)
N_regions = 5
N_groups_per_region = 4
N_groups = N_regions * N_groups_per_region
N_obs_per_group = 20
N = N_groups * N_obs_per_group

# Simulation parameters
mu_a_region, mu_b_region = 5.0, -1.0
sigma_a_region, sigma_b_region = 1.0, 0.5
rho_region = -0.5
```

```

sigma_a_group, sigma_b_group = 0.5, 0.2
rho_group = 0.3
sigma_obs = 0.5

# Generate region effects
mu_region = np.array([mu_a_region, mu_b_region])
sigmas_region = np.array([sigma_a_region, sigma_b_region])
Rho_region = np.array([[1, rho_region], [rho_region, 1]])
Sigma_region = np.outer(sigmas_region, sigmas_region) * Rho_region
region_effects = np.random.multivariate_normal(mu_region, Sigma_region, size=N_regions)

# Generate group effects nested in regions
group_to_region = np.repeat(np.arange(N_regions), N_groups_per_region)
sigmas_group = np.array([sigma_a_group, sigma_b_group])
Rho_group = np.array([[1, rho_group], [rho_group, 1]])
Sigma_group = np.outer(sigmas_group, sigmas_group) * Rho_group
group_effects = np.zeros((N_groups, 2))
for g in range(N_groups):
    reg = group_to_region[g]
    group_effects[g] = np.random.multivariate_normal(region_effects[reg], Sigma_group)

# Generate observations
group_id = np.repeat(np.arange(N_groups), N_obs_per_group)
x = np.random.normal(0, 1, size=N).astype(np.float32)
y = np.random.normal(0, 1, size=N).astype(np.float32)

m.data_on_model = {
    "group_id": group_id.astype(np.int32),
    "group_to_region": group_to_region.astype(np.int32),
    "x": x,
    "y": y,
}

# Define model -----
def model_hierarchical(group_id, group_to_region, x, y):
    sigma = m.dist.exponential(1, name='sigma')

    # 1. Region Level
    a_reg = m.dist.normal(5, 2, shape=(5,), name='a_reg')
    b_reg = m.dist.normal(-1, 1, shape=(5,), name='b_reg')

    # 2. Group Level (Nested)

```

```

# Means are indexed from region level
a_grp = m.dist.normal(a_reg[group_to_region], 0.5, shape=(20,), name='a_grp')
b_grp = m.dist.normal(b_reg[group_to_region], 0.2, shape=(20,), name='b_grp')

# 3. Likelihood
mu = a_grp[group_id] + b_grp[group_id] * x
m.dist.normal(mu, sigma, obs=y)

# Run sampler -----
m.fit(model_hierarchical, num_samples=1000, num_warmup=500, num_chains=1, progress_bar=False)

# Diagnostic -----
import io
from contextlib import redirect_stdout
with redirect_stdout(io.StringIO()):
    m.summary() # Populates m.tab_summary

print("\n### Parameter Comparison")
print(f"{'Parameter':<15} | {'True':<10} | {'Posterior Mean':<15}")
print("-" * 45)
params_to_check = {
    "sigma": 0.5, # true sigma_obs
}

for p, true_val in params_to_check.items():
    if p in m.tab_summary.index:
        post_mean = m.tab_summary.loc[p, "mean"]
        print(f"{p:<15} | {true_val:<10.2f} | {post_mean:<15.2f}")

```

jax.local_device_count 16

arviz - WARNING - Shape validation failed: input_shape: (1, 1000), minimum_shape: (chains=2,

```

### Parameter Comparison
Parameter      | True      | Posterior Mean
-----
sigma          | 0.50      | 0.98

```

Julia

```
using BayesianInference

# Setup device-----
m = importBI(platform="cpu")

# Define model -----
@BI function model(group_id, region_id, x, y)
    sigma = m.dist.exponential(1, name="sigma")

    # Region Level
    a_reg, b_reg = m.effects.varying_effects(
        N_vars = 1,
        N_group = 5,
        group_id = region_id,
        group_name = "region"
    )

    # Group Level (Nested)
    a_grp, b_grp = m.effects.varying_effects(
        N_vars = 1,
        N_group = 20,
        group_id = group_id,
        alpha_bar = a_reg,
        beta_bar = b_reg,
        group_name = "group"
    )

    mu = a_grp .+ b_grp .* x
    m.dist.normal(mu, sigma, obs=y)
end

# Run mcmc -----
m.fit(model)
m.summary()
```

R

```
library(BayesianInference)
jnp = reticulate::import('jax.numpy')
```



```

# setup platform-----
m=importBI(platform='cpu')

# Define model -----
model <- function(group_id, region_id, x, y){
  sigma = bi.dist.exponential(1, name='sigma')

  # Region level
  region_veff = m$effects$varying_effects(
    N_vars = 1L,
    N_group = 5L,
    group_id = region_id,
    group_name = 'region'
  )
  a_reg = region_veff[[1]]
  b_reg = region_veff[[2]]

  # Group level (Nested)
  group_veff = m$effects$varying_effects(
    N_vars = 1L,
    N_group = 20L,
    group_id = group_id,
    alpha_bar = a_reg,
    beta_bar = b_reg,
    group_name = 'group'
  )
  a_grp = group_veff[[1]]
  b_grp = group_veff[[2]]

  mu = a_grp + b_grp * x
  m$dist$normal(mu, sigma, obs=y)
}

# Run MCMC -----
m$fit(model)
m$summary()

```

Mathematical Details

We model the outcome Y_i for observation i . The intercept α_j and slope β_j for group $j(i)$ are nested within region $k(j)$.

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha_{[j(i)]} + \beta_{[j(i)]} X_i$$

The vector of group-level effects follows a Multivariate Normal distribution centered on the region-level effects:

$$\begin{bmatrix} \alpha_j \\ \beta_j \end{bmatrix} \sim \text{MVNormal} \left(\begin{bmatrix} \gamma_{\alpha, [k(j)]} \\ \gamma_{\beta, [k(j)]} \end{bmatrix}, \Sigma_{group} \right)$$

The region-level effects themselves follow their own Multivariate Normal distribution:

$$\begin{bmatrix} \gamma_{\alpha, k} \\ \gamma_{\beta, k} \end{bmatrix} \sim \text{MVNormal} \left(\begin{bmatrix} \bar{\gamma}_{\alpha} \\ \bar{\gamma}_{\beta} \end{bmatrix}, \Sigma_{region} \right)$$

Where:

- $\Sigma = \mathbf{SRS}$ (diagonal matrix of standard deviations $\mathbf{S}$ and correlation matrix $\mathbf{R}$).
- $\bar{\gamma}_{\alpha}, \bar{\gamma}_{\beta}$ are the global average intercept and slope.
- $\Sigma_{region}, \Sigma_{group}$ are the covariance matrices for each hierarchical level.

Notes

Note

- By nesting the **MVN** prior, we allow the model to learn how the correlation between intercepts and slopes persists across different levels of the hierarchy.
- The non-centered parameterization (used in the `m.effects.varying_effects` built-in function) is critical for convergence in these complex models.
- This approach can be generalized to N levels and M variables by increasing the dimensionality of the vectors and matrices.

Reference(s)